40

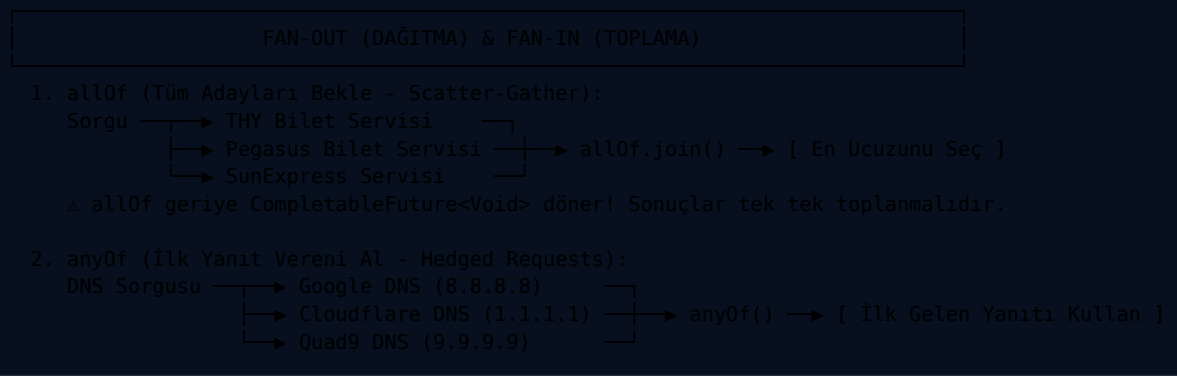
BÖLÜM: ASENKRON BORU HATLARI & COMPLETABLEFUTURE | MODULE: ASYNCHRONOUS PIPELINES & COMPLETABLEFUTURE

Fan-Out & Fan-In Orkestrasyonu: CompletableFuture.allOf vs anyOf ile Dağıtık Koordinasyon

Fan-Out & Fan-In Coordination: CompletableFuture.allOf vs anyOf

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

allOf: Bir otobüs şoförünün tüm yolcuların (uçak bileti arayan 5 farklı sitenin) binmesini beklemesi gibidir. Herkes binmeden otobüs kalkmaz; sonra yolcuların biletlerini sayarsın. anyOf ise bir taksi durağında sıraya girip ilk gelen boş taksiye binmektir. 3 taksiyi birden çağırırsın, kapıya ilk yanaşan taksiye biner gidersin; diğerleri boşa çıkar!

🇬🇧 ENGLISH (ELI5)

`CompletableFuture.allOf` is scatter-gather: A travel aggregator querying 5 airline APIs concurrently. The master request waits until every single airline responds before filtering the lowest price. `CompletableFuture.anyOf` is hedged requests: Querying 3 redundant DNS mirrors (Google, Cloudflare, Quad9) simultaneously and immediately returning whichever responds first, discarding trailing packets.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

HotSpot allOf ve anyOf İç Mantığı: 1) `allOf(CompletableFuture\<?>... cfs)`: Dahili olarak bir `BiRelay` ağacı kurar. Her bir future tamamlandığında bir sayaç eksiltilir. Sayaç 0'a ulaştığında bileşik future tetiklenir. DÖNÜŞ TİPİ `CompletableFuture\<Void>`'dir! Sonuçları almak için orijinal future dizisi stream edilip `CompletableFuture::join` ile toplanır (bu noktada tümü bittiği için join ASLA bloklamaz). 2) `anyOf(CompletableFuture\<?>... cfs)`: İlk tamamlanan (veya ilk istisna fırlatan) future bileşik nesneyi çözer. DÖNÜŞ TİPİ `CompletableFuture\<Object>`'tir ve tip dönüşümü (cast) gerektirir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

HotSpot Implementation Details: 1) `allOf`: Constructs a binary tree of completion relays. Each completing leaf decrements an atomic countdown latch. Returns `CompletableFuture\<Void>` by design to accommodate heterogeneous return types. To extract payloads, stream over the input list mapping via `CompletableFuture::join`—which is guaranteed non-blocking because all tasks are already resolved. 2) `anyOf`: Binds an identical completion handler to all upstream stages; the first node to transition invokes `completeRelay()`. Returns `CompletableFuture\<Object>`, requiring explicit downcasting.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: allOf ile Scatter-Gather Fiyat Toplama
import java.util.List;
import java.util.concurrent.CompletableFuture;

public class ScatterGatherAggregator {
    record PriceQuote(String provider, double price) {}

    static CompletableFuture<PriceQuote> fetchQuote(String provider, double price, long delayMs) {
        return CompletableFuture.supplyAsync(() -> {
            try { Thread.sleep(delayMs); } catch (InterruptedException _) {}
            return new PriceQuote(provider, price);
        });
    }

    void main() {
        var futures = List.of(
            fetchQuote("Airline-A", 180.0, 50),
            fetchQuote("Airline-B", 150.0, 80),
            fetchQuote("Airline-C", 210.0, 30)
        );

        // allOf ile tüm servislerin bitmesini bekle
        CompletableFuture<Void> allDone = CompletableFuture.allOf(
            futures.toArray(new CompletableFuture[0])
        );

        // Non-blocking toplama (join garantili olarak anında döner!)
        CompletableFuture<List<PriceQuote>> quotesFuture = allDone.thenApply(_ ->
            futures.stream().map(CompletableFuture::join).toList()
        );

        List<PriceQuote> quotes = quotesFuture.join();
        PriceQuote bestQuote = quotes.stream()
            .min((q1, q2) -> Double.compare(q1.price(), q2.price()))
            .orElseThrow();

        System.out.println("En İyi Teklif: " + bestQuote);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

allOf içinde tek bir future patlarsa (exception), allOf da anında patlar! Ancak diğer çalışan future'lar arka planda çalışmaya ve kaynak tüketmeye devam eder. Dağıtık sistemlerde allOf kullanırken her alt task'ın içine 'exceptionally' koyarak default fallback üretmesini sağlayın.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

If any child future fails inside `allOf`, the composite future fails immediately. However, surviving peer futures continue running in the background, squandering resources. Always attach `.exceptionally()` or fallback handlers to individual stages before passing them into `allOf`.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• allOf tüm görevlerin bitmesini bekler (scatter-gather, Void döner). anyOf ilk biteni alır (hedged request, Object döner).

🎯 Key Takeaways for the Interview

• `allOf` waits for all stages to conclude (scatter-gather, returns `Void`). `anyOf` yields the first finished stage (hedged calls, returns `Object`).